

Evolution Strategies

This PDF conversion (v1) was generated on October 10, 2025*

Online version:

<https://lilianweng.github.io/posts/2019-09-05-evolution-strategies/>

Date: **Estimated Reading Time:** min **Author:** Lilian Weng

⁰This file was prepared by Sakura (bili_sakura@zju.edu.cn). Please contact for any issues or copyright concerns. The original source of this file is <https://lilianweng.github.io/posts/2019-09-05-evolution-strategies/>.

Contents

1	Evolution Strategies	3
2	What are Evolution Strategies?#	4
3	Simple Gaussian Evolution Strategies#	5
4	Covariance Matrix Adaptation Evolution Strategies (CMA-ES)#	5
4.1	Updating the Mean#	7
4.2	Controlling the Step Size#	7
4.3	Adapting the Covariance Matrix#	8
9	Citation	9
5	Natural Evolution Strategies#	10
5.1	Natural Gradients#	10
5.2	Estimation using Fisher Information Matrix#	10
5.3	NES Algorithm#	10
6	Applications: ES in Deep Reinforcement Learning#	10
6.1	OpenAI ES for RL#	10
6.2	Exploration with ES#	10
6.3	CEM-RL#	10
7	Extension: EA in Deep Learning#	10
7.1	Hyperparameter Tuning: PBT#	10
7.2	Network Topology Optimization: WANN#	10
8	References#	10
10	References	11

[Lil'Log](#)

- [|](#)
- [Posts](#)
- [Archive](#)
- [Search](#)
- [Tags](#)
- [FAQ](#)

1 Evolution Strategies

Date: September 5, 2019 | Estimated Reading Time: 22 min | Author: Lilian Weng
Table of Contents

- [What are Evolution Strategies?](#)
- [Simple Gaussian Evolution Strategies](#)
- [Covariance Matrix Adaptation Evolution Strategies \(CMA-ES\)](#)
 - [Updating the Mean](#)
 - [Controlling the Step Size](#)
 - [Adapting the Covariance Matrix](#)
- [Natural Evolution Strategies](#)
 - [Natural Gradients](#)
 - [Estimation using Fisher Information Matrix](#)
 - [NES Algorithm](#)
- [Applications: ES in Deep Reinforcement Learning](#)
 - [OpenAI ES for RL](#)
 - [Exploration with ES](#)
 - [CEM-RL](#)
- [Extension: EA in Deep Learning](#)
 - [Hyperparameter Tuning: PBT](#)
 - [Network Topology Optimization: WANN](#)
- [References](#)

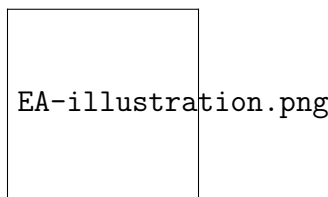


Figure 1: How natural selection works. (Image source: Khan Academy: [Darwin, evolution, & natural selection](#))

Stochastic gradient descent is a universal choice for optimizing deep learning models. However, it is not the only option. With black-box optimization algorithms, you can evaluate a target function $f(x): \mathbb{R}^n \rightarrow \mathbb{R}$, even when you don't know the precise analytic form of $f(x)$ and thus cannot compute gradients or the Hessian matrix. Examples of black-box optimization methods include [Simulated Annealing](#), [Hill Climbing](#) and [Nelder-Mead method](#).

Evolution Strategies (ES) is one type of black-box optimization algorithms, born in the family of **Evolutionary Algorithms (EA)**. In this post, I would dive into a couple of classic ES methods and introduce a few applications of how ES can play a role in deep reinforcement learning.

2 What are Evolution Strategies?#

Evolution strategies (ES) belong to the big family of evolutionary algorithms. The optimization targets of ES are vectors of real numbers, $x \in \mathbb{R}^n$.

Evolutionary algorithms refer to a division of population-based optimization algorithms inspired by *natural selection*. Natural selection believes that individuals with traits beneficial to their survival can live through generations and pass down the good characteristics to the next generation. Evolution happens by the selection process gradually and the population grows better adapted to the environment.

Evolutionary algorithms can be summarized in the following [format](#) as a general optimization solution:

Let's say we want to optimize a function $f(x)$ and we are not able to compute gradients directly. But we still can evaluate $f(x)$ given any x and the result is deterministic. Our belief in the probability distribution over x as a good solution to $f(x)$ optimization is $p_{\theta}(x)$, parameterized by θ . The goal is to find an optimal configuration of θ .

Here given a fixed format of distribution (i.e. Gaussian), the parameter θ carries the knowledge about the best solutions and is being iteratively updated across generations.

Starting with an initial value of θ , we can continuously update θ by looping three steps as follows:

1. Generate a population of samples $D = \{(x_i, f(x_i))\}$ where $x_i \sim p_{\theta}(x)$.
2. Evaluate the "fitness" of samples in D .

3. Select the best subset of individuals and use them to update θ , generally based on fitness or rank.

In **Genetic Algorithms (GA)**, another popular subcategory of EA, x is a sequence of binary codes, $x \in \{0, 1\}^n$. While in ES, x is just a vector of real numbers, $x \in \mathbb{R}^n$.

3 Simple Gaussian Evolution Strategies#

This is the most basic and canonical version of evolution strategies. It models $p_{\theta}(x)$ as a n -dimensional isotropic Gaussian distribution, in which θ only tracks the mean μ and standard deviation σ .

$$p_{\theta} = (\mu, \sigma), p_{\theta}(x) \sim \mathcal{N}(\mu, \sigma^2 I) = \mu + \sigma \mathcal{N}(0, I)$$

The process of Simple-Gaussian-ES, given $x \in \mathcal{R}^n$:

1. Initialize $\theta = \theta(0)$ and the generation counter $t=0$
2. Generate the offspring population of size Λ by sampling from the Gaussian distribution:

$$D^{(t+1)} = \{x_i^{(t+1)} \mid x_i^{(t+1)} = \mu^{(t)} + \sigma^{(t)} y_i^{(t+1)} \mid \text{where } y_i^{(t+1)} \sim \mathcal{N}(0, I), i = 1, \dots, \Lambda\}$$

3. Select a top subset of λ samples with optimal $f(x_i)$ and this subset is called **elite** set. Without loss of generality, we may consider the first k samples in $D^{(t+1)}$ to belong to the elite group — Let's label them as

$$D^{(t+1)}_{\text{elite}} = \{x_i^{(t+1)} \mid x_i^{(t+1)} \in D^{(t+1)}, i=1, \dots, \lambda, \lambda \leq \Lambda\}$$

4. Then we estimate the new mean and std for the next generation using the elite set:

$$\begin{aligned} \mu^{(t+1)} &= \text{avg}(D^{(t+1)}_{\text{elite}}) = \frac{1}{\lambda} \sum_{i=1}^{\lambda} x_i^{(t+1)} \\ \sigma^{(t+1)^2} &= \text{var}(D^{(t+1)}_{\text{elite}}) = \frac{1}{\lambda} \sum_{i=1}^{\lambda} (x_i^{(t+1)} - \mu^{(t)})^2 \end{aligned}$$

5. Repeat steps (2)-(4) until the result is good enough

4 Covariance Matrix Adaptation Evolution Strategies (CMA-ES)#

The standard deviation σ accounts for the level of exploration: the larger σ the bigger search space we can sample our offspring population. In **vanilla ES**, $\sigma^{(t+1)}$

is highly correlated with $\sigma^{\{t\}}$, so the algorithm is not able to rapidly adjust the exploration space when needed (i.e. when the confidence level changes).

CMA-ES, short for “*Covariance Matrix Adaptation Evolution Strategy*”, fixes the problem by tracking pairwise dependencies between the samples in the distribution with a covariance matrix C . The new distribution parameter becomes:

$$\theta = (\mu, \sigma, C); p_{\theta}(x) \sim \mathcal{N}(\mu, \sigma^2 C) \\ \sim \mu + \sigma \mathcal{N}(0, C)$$

where σ controls for the overall scale of the distribution, often known as *step size*.

Before we dig into how the parameters are updated in CMA-ES, it is better to review how the covariance matrix works in the multivariate Gaussian distribution first. As a real symmetric matrix, the covariance matrix C has the following nice features (See [proof](#) & [proof](#)):

- It is always diagonalizable.
- Always positive semi-definite.
- All of its eigenvalues are real non-negative numbers.
- All of its eigenvectors are orthogonal.
- There is an orthonormal basis of $\mathbb{R}^n$ consisting of its eigenvectors.

Let the matrix C have an *orthonormal* basis of eigenvectors $B = [b_1, \dots, b_n]$, with corresponding eigenvalues $\lambda_1^2, \dots, \lambda_n^2$. Let $D = \text{diag}(\lambda_1, \dots, \lambda_n)$.

$$C = B^{\top} D^2 B = \begin{bmatrix} b_1 & b_2 & \dots & b_n \end{bmatrix} \begin{bmatrix} \lambda_1^2 & 0 & \dots & 0 \\ 0 & \lambda_2^2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \lambda_n^2 \end{bmatrix} \begin{bmatrix} b_1^{\top} \\ b_2^{\top} \\ \vdots \\ b_n^{\top} \end{bmatrix}$$

The square root of C is:

$$C^{\frac{1}{2}} = B^{\top} D B$$

Symbol Meaning

$x_i^{\{t\}} \in \mathbb{R}^n$ the i -th samples at the generation (t)

$y_i^{\{t\}} \in \mathbb{R}^n$ $x_i^{\{t\}} = \mu^{\{t-1\}} + \sigma^{\{t-1\}} y_i^{\{t\}}$

$\mu^{\{t\}}$ mean of the generation (t)

$\sigma^{\{t\}}$ step size

$C^{\{t\}}$ covariance matrix

$B^{\{t\}}$ a matrix of C 's eigenvectors as row vectors

$D^{\{t\}}$ a diagonal matrix with C 's eigenvalues on the diagonal.

$p_{\sigma}^{\{t\}}$ evaluation path for σ at the generation (t)

$p_c^{\{t\}}$ evaluation path for C at the generation (t)

α_{μ} learning rate for μ 's update

α_{σ} learning rate for p_{σ}

d_{σ} damping factor for σ 's update

α_{cp} learning rate for p_c

$\alpha_{c\lambda}$ learning rate for C 's rank-min(n) update

α_{c1} learning rate for C 's rank-1 update

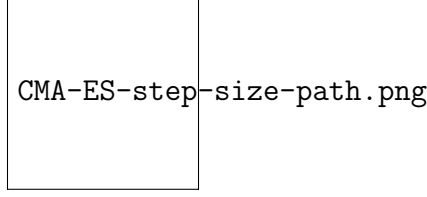


Figure 2: Three scenarios of how single steps are correlated in different ways and their impacts on step size update. (Image source: additional annotations on Fig 5 in [CMA-ES tutorial](#) paper)

4.1 Updating the Mean#

$$\mu^{(t+1)} = \mu^{(t)} + \alpha_{\mu} \frac{1}{\lambda} \sum_{i=1}^{\lambda} (x_i^{(t+1)} - \mu^{(t)})$$

CMA-ES has a learning rate $\alpha_{\mu} \leq 1$ to control how fast the mean μ should be updated. Usually it is set to 1 and thus the equation becomes the same as in vanilla ES, $\mu^{(t+1)} = \frac{1}{\lambda} \sum_{i=1}^{\lambda} x_i^{(t+1)}$.

4.2 Controlling the Step Size#

The sampling process can be decoupled from the mean and standard deviation:

$$x_i^{(t+1)} = \mu^{(t)} + \sigma^{(t)} y_i^{(t+1)} \text{, where } y_i^{(t+1)} = \frac{x_i^{(t+1)} - \mu^{(t)}}{\sigma^{(t)}} \sim \mathcal{N}(0, C)$$

The parameter σ controls the overall scale of the distribution. It is separated from the covariance matrix so that we can change steps faster than the full covariance. A larger step size leads to faster parameter update. In order to evaluate whether the current step size is proper, CMA-ES constructs an *evolution path* p_{σ} by summing up a consecutive sequence of moving steps, $\frac{1}{\lambda} \sum_{i=1}^{\lambda} y_i^{(t+1)}$, $j=1, \dots, t$. By comparing this path length with its expected length under random selection (meaning single steps are uncorrelated), we are able to adjust σ accordingly (See Fig. 2).

Each time the evolution path is updated with the average of moving step y_i in the same generation.

$$\begin{aligned} \frac{1}{\lambda} \sum_{i=1}^{\lambda} y_i^{(t+1)} &= \frac{1}{\lambda} \sum_{i=1}^{\lambda} \frac{x_i^{(t+1)} - \mu^{(t)}}{\sigma^{(t)}} = \frac{\mu^{(t+1)} - \mu^{(t)}}{\sigma^{(t)}} \\ &\sim \frac{1}{\lambda} \mathcal{N}(0, \lambda C^{(t)}) \sim \frac{1}{\sqrt{\lambda}} \mathcal{N}(0, C^{(t)}) \end{aligned}$$

Thus $\frac{1}{\sqrt{\lambda}} \mathcal{N}(0, C^{(t)}) \sim \mathcal{N}(0, I)$

By multiplying with $C^{-\frac{1}{2}}$, the evolution path is transformed to be independent of its direction. The term $\{C^{(t)}\}^{-\frac{1}{2}} = \{B^{(t)}\}^{\top} \{D^{(t)}\}^{-\frac{1}{2}} \{B^{(t)}\}$ transformation works as follows:

1. $\{B^{(t)}\}$ contains row vectors of C 's eigenvectors. It projects the original space onto the perpendicular principal axes.
2. Then $\{D^{(t)}\}^{-\frac{1}{2}} = \text{diag}(\frac{1}{\lambda_1}, \dots, \frac{1}{\lambda_n})$ scales the length of principal axes to be equal.

3. $\mathbf{B}^{\text{top}}$ transforms the space back to the original coordinate system.

In order to assign higher weights to recent generations, we use polyak averaging to update the evolution path with learning rate α_{σ} . Meanwhile, the weights are balanced so that $\mathbf{p}_{\sigma}$ is [conjugate](#), $\sim \mathcal{N}(0, \mathbf{I})$ both before and after one update.

$$\begin{aligned} \mathbf{p}_{\sigma}^{(t+1)} &= (1 - \alpha_{\sigma}) \mathbf{p}_{\sigma}^{(t)} + \sqrt{1 - (1 - \alpha_{\sigma})^2} \sqrt{\lambda} \mathbf{C}^{(t)^{-\frac{1}{2}}} \frac{\mu^{(t+1)} - \mu^{(t)}}{\sigma^{(t)}} \\ &= (1 - \alpha_{\sigma}) \mathbf{p}_{\sigma}^{(t)} + \sqrt{c_{\sigma}} (2 - \alpha_{\sigma}) \sqrt{\lambda} \mathbf{C}^{(t)^{-\frac{1}{2}}} \frac{\mu^{(t+1)} - \mu^{(t)}}{\sigma^{(t)}} \end{aligned}$$

The expected length of $\mathbf{p}_{\sigma}$ under random selection is $\mathbb{E}[\|\mathcal{N}(0, \mathbf{I})\|]$, that is the expectation of the L2-norm of a $\mathcal{N}(0, \mathbf{I})$ random variable. Following the idea in Fig. 2, we adjust the step size according to the ratio of $\|\mathbf{p}_{\sigma}^{(t+1)}\| / \mathbb{E}[\|\mathcal{N}(0, \mathbf{I})\|]$:

$$\begin{aligned} \ln \sigma^{(t+1)} &= \ln \sigma^{(t)} + \frac{\alpha_{\sigma}}{d_{\sigma}} \left(\frac{\|\mathbf{p}_{\sigma}^{(t+1)}\|}{\mathbb{E}[\|\mathcal{N}(0, \mathbf{I})\|]} - 1 \right) \\ &= \sigma^{(t)} \exp \left(\frac{\alpha_{\sigma}}{d_{\sigma}} \left(\frac{\|\mathbf{p}_{\sigma}^{(t+1)}\|}{\mathbb{E}[\|\mathcal{N}(0, \mathbf{I})\|]} - 1 \right) \right) \end{aligned}$$

where $d_{\sigma} \approx 1$ is a damping parameter, scaling how fast $\ln \sigma$ should be changed.

4.3 Adapting the Covariance Matrix#

For the covariance matrix, it can be estimated from scratch using $\mathbf{y}_i$ of elite samples (recall that $\mathbf{y}_i \sim \mathcal{N}(0, \mathbf{C})$):

$$\mathbf{C}_{\lambda}^{(t+1)} = \frac{1}{\lambda} \sum_{i=1}^{\lambda} \mathbf{y}_i^{(t+1)} \mathbf{y}_i^{(t+1)\top} = \frac{1}{\lambda} \sum_{i=1}^{\lambda} (\mathbf{x}_i^{(t+1)} - \mu^{(t)}) (\mathbf{x}_i^{(t+1)} - \mu^{(t)})^{\top}$$

The above estimation is only reliable when the selected population is large enough. However, we do want to run *fast* iteration with a *small* population of samples in each generation. That's why CMA-ES invented a more reliable but also more complicated way to update $\mathbf{C}$. It involves two independent routes,

- *Rank-min(, n) update*: uses the history of $\{\mathbf{C}_{\lambda}\}$, each estimated from scratch in one generation.
- *Rank-one update*: estimates the moving steps $\mathbf{y}_i$ and the sign information from the history.

The first route considers the estimation of $\mathbf{C}$ from the entire history of $\{\mathbf{C}_{\lambda}\}$. For example, if we have experienced a large number of generations, $\mathbf{C}^{(t+1)} \approx \text{avg}(\mathbf{C}_{\lambda}^{(i)}; i=1, \dots, t)$ would be a good estimator. Similar to $\mathbf{p}_{\sigma}$, we also use polyak averaging with a learning rate to incorporate the history:

$$\begin{aligned} \mathbf{C}^{(t+1)} &= (1 - \alpha_{\lambda}) \mathbf{C}^{(t)} + \alpha_{\lambda} \mathbf{C}_{\lambda}^{(t+1)} \\ &= (1 - \alpha_{\lambda}) \mathbf{C}^{(t)} + \alpha_{\lambda} \frac{1}{\lambda} \sum_{i=1}^{\lambda} \mathbf{y}_i^{(t+1)} \mathbf{y}_i^{(t+1)\top} \end{aligned}$$

A common choice for the learning rate is $\alpha_{\lambda} \approx \min(1, \lambda/n^2)$.

The second route tries to solve the issue that $y_i y_i^{\text{top}} = (-y_i)(-y_i)^{\text{top}}$ loses the sign information. Similar to how we adjust the step size σ , an evolution path p_c is used to track the sign information and it is constructed in a way that p_c is conjugate, $\sim \mathcal{N}(0, C)$ both before and after a new generation.

We may consider p_c as another way to compute $\text{avg}_i(y_i)$ (notice that both $\sim \mathcal{N}(0, C)$) while the entire history is used and the sign information is maintained. Note that we've known $\sqrt{k} \frac{\mu^{(t+1)} - \mu^{(t)}}{\sigma^{(t)}} \sim \mathcal{N}(0, C)$ in the [last section](#),

$$\begin{aligned} p_c^{(t+1)} &= (1 - \alpha_{cp}) p_c^{(t)} + \sqrt{1 - (1 - \alpha_{cp})^2} \sqrt{k} \frac{\mu^{(t+1)} - \mu^{(t)}}{\sigma^{(t)}} \\ &= (1 - \alpha_{cp}) p_c^{(t)} + \sqrt{\alpha_{cp}(2 - \alpha_{cp})} \lambda \frac{\mu^{(t+1)} - \mu^{(t)}}{\sigma^{(t)}} \end{aligned}$$

Then the covariance matrix is updated according to p_c :

$$C^{(t+1)} = (1 - \alpha_{c1}) C^{(t)} + \alpha_{c1} p_c^{(t+1)} p_c^{(t+1)\text{top}}$$

The *rank-one update* approach is claimed to generate a significant improvement over the *rank-min(, n)-update* when k is small, because the signs of moving steps and correlations between consecutive steps are all utilized and passed down through generations.

Eventually we combine two approaches together,

$$C^{(t+1)} = (1 - \alpha_{c\lambda} - \alpha_{c1}) C^{(t)} + \alpha_{c1} \underbrace{p_c^{(t+1)} p_c^{(t+1)\text{top}}}_{\text{rank-one update}} + \alpha_{c\lambda} \underbrace{\frac{1}{\lambda} \sum_{i=1}^{\lambda} y^{(t+1)}_i y^{(t+1)\text{top}}_i}_{\text{rank-min}(\lambda, n) \text{ update}}$$

9 Citation

Please cite this work as:

Weng, Lilian. “[Title](#)”. Lil ’Log (May 2025). <https://lilianweng.github.io/posts/2019-09-05-evolution-strategies/>

Or use the BibTeX citation:

```
@article{weng2025,
  title = {},
  author = {Weng, Lilian},
  journal = {lilianweng.github.io},
  year = {2025}, month = {May},
  url = {}
}
```



10 References

Tags: [tag1](#) [tag2](#)

[⟨ Title Title ⟩](#)

Share this article: [Twitter](#) | [LinkedIn](#) | [Reddit](#) | [Facebook](#) | [WhatsApp](#) | [Telegram](#)